

Rapport de benchmark

Inférence LLM : cloud vs DGX Spark

Objet : pipeline de génération automatique de rapports d'analyse de données de mobilité, piloté par LLM.
Question : héberger le modèle localement sur une DGX Spark apporte-t-il un gain face à un hébergement cloud ?
Période des mesures : 23–27 juillet 2026.

1. Résumé exécutif

Sur une requête isolée, l'inférence **cloud est environ 53× plus rapide** que l'inférence locale sur la DGX Spark, pour un modèle et une charge identiques (Llama-3.1-8B, ~1 265 tokens générés de chaque côté). Ce résultat n'est pas un défaut de configuration : il découle de la **bande passante mémoire** de la machine, facteur physique limitant.

L'intérêt de l'hébergement local ne se situe donc pas dans la vitesse, mais dans trois dimensions où il l'emporte nettement : **absence de quota**, **fiabilité** (100 % des appels aboutissent) et **confidentialité** des données. Le choix de la « meilleure solution » dépend de la priorité — latence brute, ou souveraineté et disponibilité.

2. Méthodologie

2.1 Principe

On exécute **le même pipeline**, sur **le même type de donnée**, avec **le même modèle**. La seule variable est le lieu d'hébergement de l'inférence. Ce choix garantit qu'on compare deux *infrastructures*, et non deux modèles : Groq n'édite pas de modèle, c'est un hébergeur qui sert des modèles open-source sur un matériel dédié.

	Configuration cloud	Configuration locale
Modèle	Llama-3.1-8B-Instruct	Llama-3.1-8B-Instruct (miroir NousResearch)
Hébergeur	Groq (API cloud)	DGX Spark (vLLM, GPU local)
Nom technique	llama-3.1-8b-instant	NousResearch/Meta-Llama-3.1-8B-Instruct

2.2 Matériel local

Élément	Valeur
Machine	NVIDIA DGX Spark
Puce	GB10 (Grace Blackwell), CPU ARM64
GPU	1 seul
Mémoire	128 Go unifiée (CPU + GPU)
Bande passante mémoire	273 Go/s — facteur limitant
Serveur d'inférence	vLLM 0.15.1 (tp=1, max-model-len=8192)
Empreinte modèle	14,99 Gio de poids + 20,55 Gio de cache KV

2.3 Instrumentation

Chaque appel LLM du pipeline est chronométré et enregistré dans la table **llm_calls_log** (provider, modèle, durée, tokens, succès, étape). Les deux configurations utilisant un identifiant de provider différent (Groq /

NVIDIA), leurs mesures se séparent automatiquement. Le débit brut a de plus été mesuré directement par requête HTTP (curl), hors pipeline.

3. Résultats

3.1 Débit brut (requête isolée, hors pipeline)

Configuration	Tokens	Temps	Débit
DGX Spark — Llama-3.1-8B	244	17,4 s	14,0 tok/s
DGX Spark — Qwen3-14B (écarté, cf. §5)	500	61,3 s	8,2 tok/s
Groq — Llama-3.1-8B (via pipeline)	~1 265	~1,7 s	~737 tok/s

3.2 Latence par appel, sur charge réelle du pipeline

Étape **S2** (génération de problèmes), sortie comparable (~1 265 tokens de chaque côté) :

Provider	Appels	Latence moy.	Débit	Sortie moy.
Groq (cloud)	4	1 720 ms	737 tok/s	~1 268 tokens
NVIDIA (Spark)	1	91 342 ms	13,8 tok/s	~1 261 tokens

Écart : ~53× en faveur du cloud, sur un volume de tokens quasi identique.

Étape **S3** (génération d'algorithmes / code), mesurée abondamment côté local :

Provider	Appels	Latence moy.	p50	p95	Débit
NVIDIA (Spark)	11	56 872 ms	56 940 ms	67 928 ms	13,8 tok/s
Groq (cloud)	—	n/a	—	—	—

Groq n'a jamais pu compléter S3 (cf. §3.3). Le débit local y est cependant identique à S2 (13,8 tok/s), ce qui confirme que la vitesse ne dépend pas de la tâche mais du matériel.

3.3 Fiabilité

Configuration	Taux de succès	Comportement observé
DGX Spark	100 %	aucune interruption, débit constant
Groq (gratuit)	dégradé	429 systématique dès le 2e ou 3e appel

Le tier gratuit de Groq applique une limite de tokens par minute trop basse pour ce pipeline (~1 000–1 200 tokens par appel). Dès le premier dépassement, le routeur blackliste le provider pour la session. **Conséquence : impossible de mener un run cloud complet sur le tier gratuit**, même après création d'un nouveau compte. C'est en soi un résultat du benchmark.

4. Analyse

4.1 Pourquoi le cloud est plus rapide

Générer un token impose de relire en mémoire les poids actifs du modèle. La vitesse de génération est donc plafonnée par la **bande passante mémoire**, non par la puissance de calcul brute :

débit théorique ~ bande passante mémoire ÷ taille du modèle en mémoire

La DGX Spark dispose de 273 Go/s (mémoire unifiée d'un poste de travail). Un service cloud spécialisé comme Groq exploite un matériel dédié à plusieurs To/s. Le rapport de bande passante explique, à lui seul, l'ordre de grandeur du facteur $\sim 50\times$ observé. **Ce n'est pas optimisable par configuration** : c'est une contrainte structurelle du type de machine.

4.2 Ce que la latence brute ne montre pas

- **Fiabilité** : le run cloud s'interrompt sur des quotas ; le run local va au bout. Sur un pipeline de centaines d'appels, un cloud gratuit est inexploitable de bout en bout, alors que le local termine.
- **Confidentialité** : en local, aucune donnée ne quitte la machine. Pour des données de mobilité contenant des informations personnelles (RGPD), ce critère peut primer sur la latence.
- **Débit en parallèle** : vLLM pratique le *continuous batching* — plusieurs requêtes simultanées coûtent presque le temps d'une seule. Le pipeline appelant en séquentiel, cet atout de la Spark n'apparaît pas dans les mesures (cf. §5).

5. Limites méthodologiques

#	Limite	Précision
1	Échantillons déséquilibrés	la comparaison S2 repose sur 4 appels Groq contre 1 appel NVIDIA. La stabilité du débit local (13,8 tok/s identique sur S2 et sur les 11 appels S3) rend cette mesure représentative, mais un échantillon plus large la solidifierait.
2	Aucun temps mur complet côté cloud	Groq n'ayant jamais dépassé S2, la durée totale d'une passe complète n'a pas pu être établie côté cloud.
3	Composante réseau	les appels cloud incluent un aller-retour Internet, pas le local. Avantage réel du self-hosting, mais il fait partie de l'écart mesuré.
4	Mesure séquentielle	le pipeline n'exploite pas le batching de vLLM ; le débit agrégé potentiel de la Spark sous charge parallèle n'est pas capturé.
5	Inférence non bit-à-bit identique	Groq applique ses propres optimisations (quantification, compilation). Même modèle et même taille, mais pas exactement le même calcul.
6	Machine partagée	la Spark hébergeait d'autres processus (~ 50 Go occupés, charge à 80 %) pendant les mesures, ce qui a pu introduire du bruit.

6. Conclusion et recommandation

Résultat. Pour l'inférence d'un LLM de 8 milliards de paramètres, le cloud (Groq) est **$\sim 53\times$ plus rapide par requête** que la DGX Spark, la bande passante mémoire d'un poste de travail étant le facteur limitant. Ce constat est structurel, non corrigeable par réglage.

Arbitrage. La « meilleure solution » dépend de la priorité :

Priorité dominante	Solution recommandée
Latence / débit par requête	Cloud (matériel dédié)
Volume élevé sans quota	DGX Spark (aucune limite journalière)
Confidentialité / RGPD	DGX Spark (données jamais externalisées)
Fiabilité de bout en bout	DGX Spark (100 % des appels aboutissent)
Coût à faible volume	Cloud gratuit (mais quotas très serrés)

Pour ce pipeline précis, traitant des données de mobilité potentiellement personnelles et enchaînant de longues séries d'appels, la **DGX Spark constitue le choix le plus pertinent malgré sa lenteur** : le tier cloud

gratuit est de toute façon inutilisable de bout en bout (quotas), et la confidentialité des données prime. Le cloud reste préférable pour un besoin ponctuel, interactif et sans contrainte de confidentialité.

7. Annexe — données brutes (llm_calls_log)

provider	step	n	moy ms	p50 ms	p95 ms	tok/s
Groq	s2_problems	4	1 720	1 659	1 891	737,1
NVIDIA	s2_problems	1	91 342	91 342	91 342	13,8
NVIDIA	s3_algorithms	11	56 872	56 940	67 928	13,8
NVIDIA	s3_algorithms_repair	3	~77 785	—	—	~13,8

Débit brut hors pipeline (curl) : Spark Llama-3.1-8B = **14,0 tok/s** ; Groq Llama-3.1-8B ~ **737 tok/s**.

Environnement local : DGX Spark GB10, vLLM 0.15.1, tp=1, gpu-mem-util=0.25, max-model-len=8192 ; modèle chargé en 14,99 Gio + 20,55 Gio de cache KV.